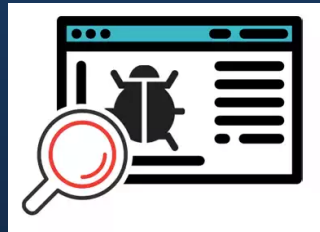


Testy jednostkowe z Microsoft Native Unit Test



„[] opanowanie teorii jest zaledwie niezbędnym wstępem”

Grzegorz Szymkowiak

Framework Microsoft Native Unit Test

Native Unit Test

- **Microsoft Native Unit Test** to wbudowany w środowisko Visual Studio framework służący do tworzenia i uruchamiania testów jednostkowych w języku C++

Native Unit Test

- Cechy:
 - **natywność dla C++:** przeznaczony specjalnie do kodu w C++, nie wymaga środowiska uruchomieniowego .NET dla samego testu
 - **Integracja z Visual Studio:** posiada wbudowane szablony projektów i jest w pełni obsługiwany przez wskaźniki oraz okno Eksplorator testów (Test Explorer)
 - **Obsługa asercji:** Udostępnia klasę Assert służącą do weryfikacji wyników

Native Unit Test

Metoda	Zastosowanie
AreEqual	porównanie wartości
AreNotEqual	sprawdzenie różnicy
IsTrue	warunek logiczny
IsFalse	negacja warunku
IsNull	sprawdzanie nullptr
IsNotNull	sprawdzanie wskaźników
Fail	wymuszenie błędu
ExpectException	test wyjątków

Przygotowanie

Przygotowanie

- Najprostszy sposób polega na przygotowaniu odrębnego pliku, który zawiera funkcję lub klasę do przetestowania. Na przykład dla funkcji sortującej bąbelkowo w C++ stworzymy dwa pliki: `BubbleSort.h`, `BubbleSort.cpp`

Przygotowanie

- Plik z rozszerzeniem `.h` w języku C++ jest plikiem nagłówkowym, który zawiera **deklarację**: funkcji, klasy, struktury itp.

```
#pragma once  
  
void BubbleSort(int A[], int n);
```


Przygotowanie

```
#include "pch.h"
#include "BubbleSort.h"

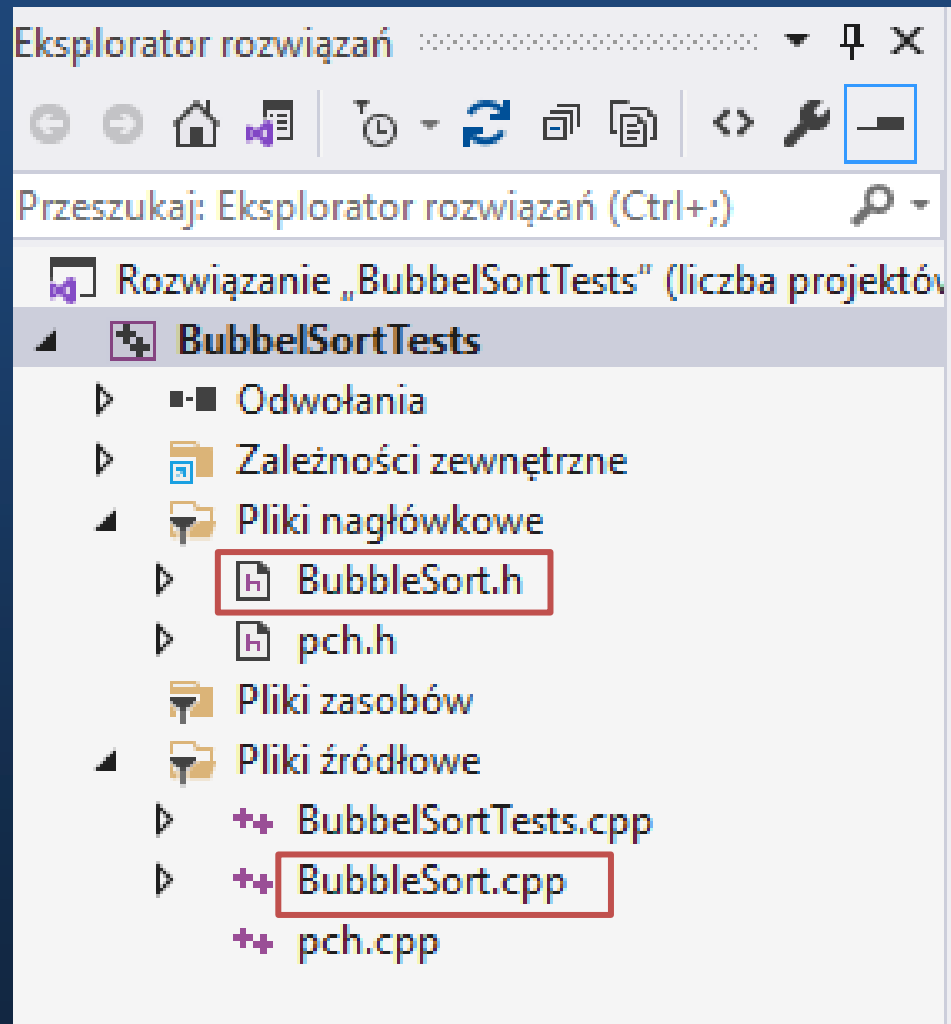
void BubbleSort(int A[], int n) {
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < n - i; j++) {
            if (A[j] > A[j + 1]) {
                int tmp = A[j];
                A[j] = A[j + 1];
                A[j + 1] = tmp;
            }
        }
    }
}
```

- Plik z rozszerzeniem `.cpp` zawiera ciało funkcji

Przygotowanie

- Oba pliki należy dołączyć do projektu testowego w odpowiednich miejscach

Przygotowanie



Nazwy klasy i metod

Nazwa metod

- Uruchomienie kreatora projektu testów jednostkowych spowoduje utworzenie klasy testowej oraz pustej metody z domyślną nazwą

Nazwa klasy i metod

```
TEST_CLASS(UnitTest1)
```

```
{
```

```
public:
```

```
    TEST_METHOD(TestMethod1)
```

```
    {
```

```
    }
```

```
    TEST_METHOD(TestMethod2)
```

```
    {
```

```
    }
```

```
};
```

Klasa z testami

Metoda testowa

Nazwa metod

- Nazwa klasy powinna odzwierciedlać testowaną funkcjonalność. Na przykład dla funkcji wykonującej sortowanie bąbelkowe zapisanej w pliku `BubbleSort.cpp` nazwy klasy testowej zazwyczaj wybiera się nazwę `BubbleSortTests`

Nazwa klasy i metod

- Nazwa metody testującej powinna opisywać wykonywaną czynność. Na przykład:
 - `SortsUnsortedArray`
 - `SortsWithDuplicates`

Nazwa klasy i metod

```
TEST_CLASS(BubbleSortTests)
{
public:
    TEST_METHOD(SortsUnsortedArray)
    {
    }

    TEST_METHOD(SortsAlreadySorted)
    {
    }
};
```

Klasa z testami

Metoda testowa

Wzorzec AAA

Wzorzec AAA

- Bardzo popularną koncepcją budowy testu jest AAA, czyli **Arrange-Act-Assert** znana również jako Given-When-Then

Wzorzec AAA

- **Arrange** - w tej części tworzy się założenia wstępne testu. Na przykład inicjalizuje się niezbędne obiekty (obiekt testowanej klasy nazywamy się **sut** lub **cut** - od **system under test** lub **class under test** - taki styl nazewnictwa pomaga odróżnić obiekt testowanej klasy, od obiektów, które tylko biorą udział w teście

Wzorzec AAA

- **Act** uruchamia testowaną funkcjonalność, a jeżeli jest taka potrzeba, to przekazujemy wymagane parametry do testowanej metody

Wzorzec AAA

- **Assert** odpowiada za sprawdzenie warunków określających powodzenie testu

Wzorzec AAA

```
// Arrange
int const N = 5;
int A[N] = { 1, 2, 3, 4, 5 };
int expected[N] = { 1, 2, 3, 4, 5 }; // Oczekiwany

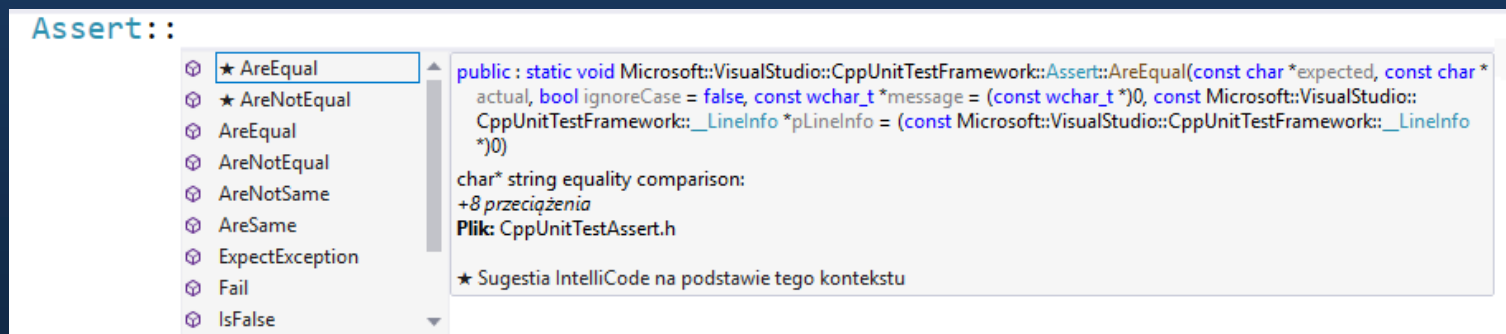
// Act
BubbleSort(A, N);

// Assert
for (int i = 0; i < N; i++) {
    Assert::AreEqual(expected[i], A[i]);
}
```

Weryfikacja poprawności testu

Weryfikacja poprawności testu

- Weryfikację poprawności testu można wykonać za pomocą klasy **Assert**, która ma wiele statycznych metod



Weryfikacja poprawności testu

- Najczęściej stosowane metody:
 - `Assert::AreEqual()` - sprawdza czy dwie wartości są równe
 - `Assert::AreNotEqual()` - sprawdza czy dwie wartości nie są równe

Weryfikacja poprawności testu

- Najczęściej stosowane metody:
 - `Assert::IsTrue()` - sprawdza, czy wyrażenie logiczne (bool) jest prawdziwe
 - `Assert::IsFalse()` - sprawdza, czy wyrażenie logiczne (bool) jest fałszywe

Weryfikacja poprawności testu

- Najczęściej stosowane metody:
 - `Assert::AreSame()` - sprawdza czy dwa obiekty to ta sama instancja
 - `Assert::AreNotSame()` - sprawdza czy dwa obiekty to różne instancje

Weryfikacja poprawności testu

- Najczęściej stosowane metody:
 - `Assert::IsNull(object)` - sprawdza, czy przekazany obiekt ma wartość null
 - `Assert::IsNotNull(object)` - sprawdzania, czy przekazany obiekt nie jest nullptr (czy istnieje jakaś instancja)

www.edunet.tychy.pl

KONIEC

Grzegorz Szymkowiak